# PMG Unified Panel Spec

**Session Date:** 2026-05-19

**Status:** Ready for implementation

**Rule #1: Nothing is removed. All existing tools, fields, tabs, and functionality stay exactly as they are. You are adding structure and new content only.**

---

## Pre-Execution Checklist

Before writing a single line of code, Replit MUST verify:

- [ ] `pmg-business-mode.js` exists at `/public/scripts/pmg-business-mode.js` and controls the Growth Mode drawer
- [ ] `pmg-expert-center.js` exists at `/public/scripts/pmg-expert-center.js` and controls the Expert Command Center
- [ ] `pmg-quick-chips.js` exists at `/public/scripts/pmg-quick-chips.js` and controls the Quick Add chip row
- [ ] `pmg-business-mode.css` exists at `/public/styles/pmg-business-mode.css`
- [ ] The `buildDrawer()` function in `pmg-business-mode.js` renders `#pmg-bm-drawer`
- [ ] The `TOOLS` array in `pmg-business-mode.js` contains all 17 tools
- [ ] The `TABS` array in `pmg-expert-center.js` contains exactly 5 tabs: `diagnose`, `engineer`, `tune`, `variations`, `save`
- [ ] The `TUNE_SLIDERS` array in `pmg-expert-center.js` contains exactly 6 sliders: `warmth`, `depth`, `formality`, `risk`, `creativity`, `length`
- [ ] The `PANELS.image.chips` array in `pmg-quick-chips.js` contains exactly one chip: `leica-look`
- [ ] All three script tags in `app.html` have version query strings that must be bumped after changes

**Do not proceed if any of the above checks fail. Report back first.**

---

## Spec 3 — Growth Mode Restructure

**File:** `artifacts/promptmegood/public/scripts/pmg-business-mode.js`

**CSS File:** `artifacts/promptmegood/public/styles/pmg-business-mode.css`

**Version tag to bump in `app.html` :** `?v=growth-mode-tabs-2` → `?v=growth-mode-goal-groups-1`

**Same bump on the CSS link tag.**

## What changes

Nothing is removed. All 17 tools stay. All fields stay. All `build()` functions stay. The `appendPersona()` function stays. The `onBuild()` function stays. The `dispatchToText()` function stays.

**What changes:**

1. The two-lane tab switcher ( `Business` / `Creator` ) is removed from the drawer UI.
2. The `buildDrawer()` function's `drawer.innerHTML` is rewritten to replace the two `pmg-bm-lane` divs with five goal-group `<details>` sections.
3. Every individual `Build Prompt →` button ( `pmg-bm-build` ) is removed from `renderTool()` .
4. A single sticky `Build My Prompt` button is added at the bottom of the drawer body.
5. A shared `About Your Brand` section is pinned at the top of the drawer body.

## The 5 Goal Groups and their tool IDs

| Group Header | Tool IDs (from TOOLS array) |
|---|---|
| **Build an Audience** | `hooks` , `brand-voice` , `bio` |
| **Sell Something** | `funnel` , `objection` , `offer` , `pricing` |
| **Create Content** | `calendar` , `series` , `repurpose` , `titles` , `cr-multidrop` , `biz-multidrop` |
| **Launch Something** | `email-seq` , `collab` |
| **Define Your Brand** | `persona` , `creator-profile` |

> Note: `pillar` (the shared lane tool) moves into **Create Content** as the last item in that group.

## Shared "About Your Brand" section

This section sits at the very top of `#pmg-bm-drawer` , above all five goal groups. It contains the existing Brand Voice fields ( `pmg-bm-audience` , `pmg-bm-tone` ) and Creator Profile fields ( `pmg-bm-cr-niche` , `pmg-bm-cr-platform` , `pmg-bm-cr-vibe` ) — all in one compact card. These

fields already persist to `localStorage` via `wireBrandVoice()` and `wireCreatorProfile()`. Those functions stay unchanged. The section heading reads:

```
About Your Brand
Fill in once. Applied to every prompt you build.
```

## Single "Build My Prompt" button

A single `<button>` with class `pmg-bm-build-all` sits at the bottom of `#pmg-bm-body`, outside all the goal groups, always visible. When clicked, it calls `onBuild(activeToolId)` where `activeToolId` is the `id` of whichever `<details>` section is currently `open`. If no section is open, it shows the error: `"Open a tool above and fill in its fields first."` The `onBuild()` function itself is unchanged — it still calls `appendPersona()` and `dispatchToText()`.

## CSS changes in `pmg-business-mode.css`

- Remove all `.pmg-bm-tabs` styles (the tab switcher is gone).
- Remove all `.pmg-bm-lane` styles (the lane divs are gone).
- Add `.pmg-bm-brand-header` styles: a compact card at the top of the drawer with a subtle border and `padding: 14px 16px`.
- Add `.pmg-bm-goal-group` styles: a `<details>` element with a bold `<summary>` and `padding: 0 0 8px`.
- Add `.pmg-bm-build-all` styles: a full-width primary button, `position: sticky; bottom: 0`, matching the existing `.pmg-bm-build` visual style but wider.

## New drawer HTML structure (pseudocode — Replit writes the actual code)

```
#pmg-bm-drawer
  .pmg-bm-head          ← unchanged (title + close button)
  .pmg-bm-body
    .pmg-bm-brand-header  ← NEW: About Your Brand (existing fields, rewired)
    <details class="pmg-bm-goal-group">
      <summary>Build an Audience</summary>
      [renderTool('hooks') without Build button]
      [renderTool('brand-voice') without Build button]
      [renderTool('bio') without Build button]
    </details>
```

```
<details class="pmg-bm-goal-group">
  <summary>Sell Something</summary>
  [renderTool('funnel') without Build button]
  [renderTool('objection') without Build button]
  [renderTool('offer') without Build button]
  [renderTool('pricing') without Build button]
</details>
<details class="pmg-bm-goal-group">
  <summary>Create Content</summary>
  [renderTool('calendar') without Build button]
  [renderTool('series') without Build button]
  [renderTool('repurpose') without Build button]
  [renderTool('titles') without Build button]
  [renderTool('cr-multidrop') without Build button]
  [renderTool('biz-multidrop') without Build button]
  [renderTool('pillar') without Build button]
</details>
<details class="pmg-bm-goal-group">
  <summary>Launch Something</summary>
  [renderTool('email-seq') without Build button]
  [renderTool('collab') without Build button]
</details>
<details class="pmg-bm-goal-group">
  <summary>Define Your Brand</summary>
  [renderTool('persona') without Build button]
  [renderTool('creator-profile') without Build button]
</details>
<button class="pmg-bm-build-all">Build My Prompt</button>
```

## Post-execution verification

- [ ] Opening Growth Mode shows the "About Your Brand" section at the top with all 6 existing fields
- [ ] Five collapsible goal groups are visible, each with a bold header
- [ ] Clicking a group header expands it to show its tools
- [ ] No individual "Build Prompt →" buttons exist anywhere in the drawer
- [ ] A single "Build My Prompt" button is visible at the bottom
- [ ] Clicking "Build My Prompt" with a tool expanded and fields filled fires `onBuild()`, closes the drawer, switches to Text panel, and populates `#goal`
- [ ] Brand Voice and Creator Profile fields still persist to localStorage
- [ ] The `appendPersona()` function still appends brand context to the prompt

# Spec 4 — ECC Redesign

**File:** `artifacts/promptmegood/public/scripts/pmg-expert-center.js`

**Version tag to bump in `app.html` :** `?v=ecc-pt2-2-topbar` → `?v=ecc-insider-tab-1`

## What changes

Nothing is removed. All existing tabs stay. All existing fields stay. All existing functionality stays. The `buildTunedPrompt()` function stays. The `buildEngineeredPrompt()` function stays. The `renderDiagnose()` , `renderEngineer()` , `renderVariations()` , and `renderSave()` functions stay completely unchanged.

**What changes:**

1. Tab labels are renamed in the `TABS` array (IDs stay the same — do NOT change IDs).
2. A new sixth tab is inserted between `engineer` and `tune` .
3. The `renderTune()` function gets updated slider label rendering.

## Tab renames — change `label` only, never `id`

| Current `id` | Current `label` | New `label` |
|---|---|---|
| `diagnose` | `Diagnose` | `Find Weak Spots` |
| `engineer` | `Architect` | `Correct the AI` |
| `tune` | `Tune` | `Set the Guardrails` |
| `variations` | `Variations` | `Variations` ← unchanged |
| `save` | `Save` | `Vault` |

## New Tab 2 — "Add What Only You Know"

Insert a new entry into the `TABS` array between `engineer` and `tune` :

JavaScript

```
{ id: 'insider', label: 'Add What Only You Know', help: 'Give the AI the contex
```

Add a new case to `renderActivePane()` :

JavaScript

```
else if (_activeTab === 'insider') renderInsider(pane);
```

Add a new `renderInsider(pane)` function. It renders six textarea fields in order. Each field has a label above it and a placeholder inside. The state for all six fields is stored under `_state.insider` (same pattern as `_state.tune` and `_state.engineer`). Initialize `_state.insider` in the state bootstrap block alongside the existing `if (!_state.engineer)` and `if (!_state.tune)` guards.

The six fields:

| Field key | Label | Placeholder |
|---|---|---|
| `insiderContext` | `Insider Context` | `Things only you know — internal names, proprietary process names, jargon your audience uses that no one outside would recognize.` |
| `hardRules` | `Hard Rules` | `Absolute constraints the AI must never violate. E.g. "Never mention competitors by name." "Always end with a question."` |
| `antiPatterns` | `Anti-Patterns` | `Specific phrases or structures the AI defaults to that you hate. E.g. "Never start a sentence with 'Certainly'." "No bullet lists."` |
| `calibrationExample` | `Calibration Example` | `Paste a sample of your best previous output. The AI will match this standard.` |
| `neverSay` | `Never Say` | `Words, phrases, or topics that must never appear in the output.` |
| `alwaysInclude` | `Always Include` | `Elements, phrases, or references that must always appear in the output.` |

The `renderInsider()` function must include a CTA row at the bottom with two buttons:

- `Reset` (secondary) — clears all six fields and calls `writeState(_state)`
- `Apply These Rules` (primary) — reads all six non-empty fields, appends them as a `## Your Rules` block to the current `#goal` text, calls `showStatusPill('Your Rules Applied')`, and calls `applyAndClose()`

The format appended to the goal when "Apply These Rules" is clicked:

Plain Text

```
## Your Rules
[Insider Context: {value}]
[Hard Rules: {value}]
[Anti-Patterns: {value}]
[Calibration Example: {value}]
[Never Say: {value}]
[Always Include: {value}]
```

Only include lines where the field has a non-empty value.

## Slider label upgrade in `renderTune()`

The current slider rendering in `renderTune()` produces:

JavaScript

```
el('header', null, [el('span', null, s.left), el('span', null, s.right)])
```

This shows the left and right endpoint labels. That stays. Add two more things to each slider:

1. A label above the header showing the slider's semantic name (derived from the key: `warmth` → `Warmth`, `depth` → `Depth`, etc.)
2. A live semantic value label below the range input that updates as the user drags. The value label uses the existing `s.text(v)` function — when it returns a non-null string, display it. When it returns null (middle position), display `"Neutral"`.

Updated slider DOM structure per slider:

Plain Text

```
<div class="pmg-ec-slider">
  <span class="pmg-ec-slider-name">{Capitalized key name}</span>
  <header>
    <span>{s.left}</span>
    <span>{s.right}</span>
  </header>
  <input type="range" ...>
  <span class="pmg-ec-slider-value">{semantic value or "Neutral"}</span>
</div>
```

Add CSS for `.pmg-ec-slider-name` and `.pmg-ec-slider-value` to the `injectStyles()` function:

CSS

```
.pmg-ec-slider-name { font-weight: 600; font-size: .85rem; margin-bottom: 2px;
.pmg-ec-slider-value { display: block; font-size: .82rem; color: var(--color-pr
```

### Post-execution verification

- [ ] Expert Center opens and shows 6 tabs: `Find Weak Spots`, `Correct the AI`, `Add What Only You Know`, `Set the Guardrails`, `Variations`, `Vault`
- [ ] Tab IDs in the DOM are still `diagnose`, `engineer`, `insider`, `tune`, `variations`, `save`
- [ ] "Add What Only You Know" tab shows all 6 fields with correct labels and placeholders
- [ ] "Apply These Rules" appends a `## Your Rules` block to `#goal` and closes the drawer
- [ ] "Set the Guardrails" tab still shows all 10 behavior toggles and all 6 sliders
- [ ] Each slider now shows its name above, left/right labels, and a live semantic value below
- [ ] All other tabs (`Find Weak Spots`, `Correct the AI`, `Variations`, `Vault`) are completely unchanged
- [ ] State persists correctly to `localStorage` key `promptmegood:expertCenter:state:v1`

---

## Spec 5 — Camera Look Smart Chip

**File:** `artifacts/promptmegood/public/scripts/pmg-quick-chips.js`

**Version tag to bump in `app.html`:** `?v=chips-1` → `?v=chips-2`

### What changes

The `PANELS.image.chips` array currently has one static chip: `leica-look`. It stays as the default. The chip label and phrase displayed to the user changes dynamically based on keywords detected in the `#pmg-vs-image-goal` textarea. Zero new chips are added to the DOM. The row stays exactly the same width. One chip. Always.

### The four camera looks and their trigger keywords

| Chip ID | Label | Trigger keywords (case-insensitive, partial match) | Phrase appended to prompt |
|---|---|---|---|
| `leica-look` | `Leica Look` | **Default** (no keywords matched, or: `portrait`, | `shot on a Leica with its rich color signature, gentle film highlights, and` |

| | | | |
|---|---|---|---|
| | | face , person , people , headshot , model ) | distinctive Leica color rendition |
| hasselblad | Hasselblad | product , architecture , building , interior , car , vehicle , commercial , studio | shot on a Hasselblad with clinical medium-format sharpness, neutral color science, and maximum tonal detail |
| fujifilm | Fujifilm Film Simulation | travel , food , lifestyle , street , nature , landscape , cafe , market | shot on a Fujifilm with warm film simulation tones, Classic Chrome color rendering, and organic grain |
| contax-t2 | Contax T2 | candid , editorial , flash , party , night , documentary , snapshot | shot on a Contax T2 with its intimate candid warmth, slight softness, and celebrity-editorial grain |

## Implementation

Add a `detectChip(text)` function inside the IIFE that takes the current textarea value as a string and returns one of the four chip config objects based on keyword matching. Priority order if multiple keyword groups match: Hasselblad > Fujifilm > Contax T2 > Leica (default).

Add a `CHIP_CONFIGS` object inside the IIFE containing all four chip definitions (id, label, phrase, keywords array).

After `ensureMounted()` successfully mounts the image chip row, attach an `input` event listener to `#pmg-vs-image-goal`. On each `input` event, call `detectChip(textarea.value)` and update the visible chip button's `data-chip-id`, `textContent`, and `title` attribute to match the detected chip config. The chip button element already exists in the DOM — just update its attributes in place. Do not re-render the row.

The `appendPhrase()` function is unchanged. When the user clicks the chip, it reads `data-chip-id` from the button, looks up the phrase from `CHIP_CONFIGS`, and appends it. This already works — just make sure `CHIP_CONFIGS` is accessible inside the click handler.

## Post-execution verification

- [ ] Image Quick Add row still shows exactly one chip
- [ ] Default chip label is `Leica Look`

- [ ] Typing "portrait shoot" in the image goal field changes the chip to `Leica Look` (default, no change)
- [ ] Typing "product photography" changes the chip to `Hasselblad`
- [ ] Typing "travel food" changes the chip to `Fujifilm Film Simulation`
- [ ] Typing "candid street" changes the chip to `Contax T2`
- [ ] Clicking the chip appends the correct phrase for whichever camera look is currently shown
- [ ] Clearing the textarea resets the chip to `Leica Look`
- [ ] Video Quick Add row is completely unchanged

---

## Global Rules for This Implementation

1. **Nothing is removed.** Every existing tool, field, tab, function, and CSS class that exists today must still exist and work after this implementation.
2. **Version bump every changed file's query string** in `app.html` to bust the cache.
3. **Do not touch** `pmg-ux.js`, `pmg-visual-studio.js`, `pmg-chassis-v3.js`, `pmg-auto-boost.js`, `pmg-storyboard.js`, `pmg-adv-mirror.js`, `pmg-tune-chips.js`, `pmg-growth-actions.js`, `pmg-dna-card.js`, `pmg-suggestions.js`, `pmg-handoff.js`, `pmg-chip-tooltips.js`, or any backend file.
4. **Do not touch** `index.html`, `pricing.html`, `guide.html`, `manual.html`, or any page other than `app.html`.
5. **Run a double pass:** After implementing, re-read every changed file top to bottom and verify no existing logic was accidentally removed or broken.
6. **Report back** with a checklist of every file changed and every version string bumped before closing the task.